

meituan-longcat/LongCat-2.0 · Hugging Face

Source: <https://huggingface.co/meituan-longcat/LongCat-2.0>

[Hugging Face](#)

[Models](#)

[Datasets](#)

[Spaces](#)

[Buckets](#)

[NEW](#)

[Docs](#)

[Enterprise](#)

[Pricing](#)

[Website](#)

[Tasks](#)

[HuggingChat](#)

[Collections](#)

[Languages](#)

[Organizations](#)

[Community](#)

[Blog](#)

[Posts](#)

[Daily Papers](#)

[Learn](#)

[Discord](#)

[Forum](#)

[GitHub](#)

[Solutions](#)

[Team & Enterprise](#)

[Hugging Face PRO](#)

[Enterprise Support](#)

[Inference Providers](#)

[Inference Endpoints](#)

[Storage Buckets](#)

[Log In](#)

[Sign Up](#)

[meituan-longcat](#)

[/](#)

[LongCat-2.0](#)

[like](#)

[123](#)

[Follow](#)

[LongCat](#)

[3.45k](#)

[Text Generation](#)

[Safetensors](#)

[Transformers](#)

[LongCat-2.0](#)

[conversational](#)

[License:](#)

[mit](#)

[Model card](#)

[Files and versions](#)

xet

Community

2

Copy to bucket

NEW

Use this model

Instructions to use meituan-longcat/LongCat-2.0 with libraries, inference providers, notebooks, and local apps. Follow these links to get started.

Libraries

Transformers

How to use meituan-longcat/LongCat-2.0 with Transformers:

Use a pipeline as a high-level helper

```
from transformers import pipeline
```

```
pipe = pipeline("text-generation", model="meituan-longcat/LongCat-2.0")
```

```
messages = [
```

```
    {"role": "user", "content": "Who are you?"},
```

```
]
```

```
pipe(messages)
```

Load model directly

```
from transformers import LongcatCausalLM
```

```
model = LongcatCausalLM.from_pretrained("meituan-longcat/LongCat-2.0", dtype="auto")
```

Notebooks

Google Colab

Kaggle

Local Apps

Settings

vLLM

How to use meituan-longcat/LongCat-2.0 with vLLM:

Install from pip and serve model

Install vLLM from pip:

```
pip install vllm
```

Start the vLLM server:

```
vllm serve "meituan-longcat/LongCat-2.0"
```

Call the server using curl (OpenAI-compatible API):

```
curl -X POST "http://localhost:8000/v1/chat/completions" \
```

```
-H "Content-Type: application/json" \
```

```
--data '{
```

```
    "model": "meituan-longcat/LongCat-2.0",
```

```
    "messages": [
```

```
        {
```

```
            "role": "user",
```

```
            "content": "What is the capital of France?"
```

```
        }
```

```
    ]
```

```
}'
```

Use Docker

```
docker model run hf.co/meituan-longcat/LongCat-2.0
```

SGLang

How to use meituan-longcat/LongCat-2.0 with SGLang:

```

Install from pip and serve model
# Install SGLang from pip:
pip install sglang
# Start the SGLang server:
python3 -m sglang.launch_server \
    --model-path "meituan-longcat/LongCat-2.0" \
    --host 0.0.0.0 \
    --port 30000
# Call the server using curl (OpenAI-compatible API):
curl -X POST "http://localhost:30000/v1/chat/completions" \
    -H "Content-Type: application/json" \
    --data '{
        "model": "meituan-longcat/LongCat-2.0",
        "messages": [
            {
                "role": "user",
                "content": "What is the capital of France?"
            }
        ]
    }'
```

Use Docker images

```

docker run --gpus all \
    --shm-size 32g \
    -p 30000:30000 \
    -v ~/.cache/huggingface:/root/.cache/huggingface \
    --env "HF_TOKEN=<secret>" \
    --ipc=host \
    lmsysorg/sglang:latest \
    python3 -m sglang.launch_server \
        --model-path "meituan-longcat/LongCat-2.0" \
        --host 0.0.0.0 \
        --port 30000
# Call the server using curl (OpenAI-compatible API):
curl -X POST "http://localhost:30000/v1/chat/completions" \
    -H "Content-Type: application/json" \
    --data '{
        "model": "meituan-longcat/LongCat-2.0",
        "messages": [
            {
                "role": "user",
                "content": "What is the capital of France?"
            }
        ]
    }'
```

Docker Model Runner

How to use meituan-longcat/LongCat-2.0 with Docker Model Runner:

```
docker model run hf.co/meituan-longcat/LongCat-2.0
```

Browse Quantizations to use this model in llama.cpp, Ollama, LM Studio, or any compatible app.

LongCat-2.0

Model Introduction

We introduce LongCat-2.0, a large-scale MoE language model with 1.6 trillion total parameters and ~48 billion activated per token – a substantial step up from previous LongCat models, accompanied by several architectural improvements.

Both the full training run and the large-scale deployment are built entirely on AI ASIC superpods. Pretraining spans millions of accelerator-days across more than 35 trillion tokens, with no rollbacks or irrecoverable loss spikes – demonstrating that we have the capability to conduct frontier-scale training on alternative hardware platforms.

To strengthen the model on long-horizon tasks, we introduce LongCat Sparse Attention and train LongCat-2.0 on hundreds of billions of tokens of 1M-context data. Together with dedicated post-training, this gives LongCat-2.0 strong performance on coding and agentic tasks.

LongCat-2.0 is deeply integrated with mainstream harnesses such as Claude Code, OpenClaw, and Hermes, delivering strong performance across code understanding, repository-level edits, automated task execution, and agentic workflows – providing developers with a more stable and efficient collaborative experience.

Key Features

🌟 LongCat Sparse Attention

To address the output discontinuity and quadratic scoring bottleneck of the Lightning Indexer in DSA, we introduce LongCat Sparse Attention (LSA). LSA features three orthogonal improvements:

Streaming-aware Indexing (SI) reshapes the token selection budget to combine hardware-aligned contiguous access with dynamic random selection. This turns fragmented memory access into predictable sequential reads, achieving coalesced HBM access and high effective bandwidth.

Cross-Layer Indexing (CLI) leverages the empirical stability of attention saliency across adjacent layers to amortize indexing cost: a single indexing pass serves several consecutive layers at inference time, made possible by cross-layer distillation during training.

Hierarchical Indexing (HI) uses a coarse-to-fine, two-stage scoring scheme – first a coarse recall via block-level approximate scoring, then fine-grained token selection within the recalled candidates – shrinking the candidate space the indexer must process per query.

All strategies seamlessly extend to the 3-step Multi-Token Prediction module for speculative decoding. For CLI, the target model shares an index every 2 layers, while all 3 MTP draft steps share a single pass.

🌟 N-gram Embedding

LongCat-2.0 inherits N-gram Embedding from LongCat-Flash-Lite, improving parameter utilization efficiency by expanding parameters in sparse dimensions orthogonal to MoE. 135B N-gram Embedding parameters are included in the model, which adheres to the following scaling principles:

The sparsity of MoE has crossed the sweet spot.

The proportion of N-gram Embedding is constrained within an optimal range.

These two principles guarantee the robust superiority of N-gram Embedding compared to equivalent-sized pure MoE models.

For more details please refer to our blog.

Evaluation Results

We evaluate LongCat-2.0 against leading proprietary models across agentic, coding, search, productivity and foundational capabilities. Unless noted with *, all scores are measured in-house under a unified harness.

Benchmark

LongCat-2.0

Gemini 3.1 Pro

GPT-5.5

Claude Opus 4.6

Claude Opus 4.7

Claude Opus 4.8

Code Agent

Terminal-Bench 2.1

70.8

70.7*

73.8*

-

71.7*

78.9*

SWE-bench Pro

59.5

54.2*

58.6*

57.3*

64.3*

69.2*

SWE-bench Multilingual

77.3

76.9*

-

77.8*

80.5*

84.8*

General Agent

FORTE ↗

73.2

70.3

77.8

73.2

77.6

77.2

BrowseComp

79.9

85.9*

84.4*

84.0*

79.3*

84.3*

RWSearch ↗

78.8

76.3

85.3

81.3

79.3

77.3

Foundational

IFEval

90.0

96.1

95.0

92.2

88.7

86.0

Writing Bench

83.8

83.7

84.7

-

85.3

85.2

IMO-AnswerBench

81.8

90.0

79.5

75.3*

81.8

75.3

GPQA-diamond

88.9

94.3*

93.6*

91.3*

94.2*

92.4

Notes: * – cited from the model's official report; – – no comparable public score.

Chat Website

You can chat with LongCat-2.0 on our official website: <https://longcat.ai/>.

Deployment

LongCat-2.0 can be deployed on both GPU and NPU platforms.

GPU

We have implemented adaptations in SGLang (PR) to support the deployment of LongCat-2.0. Hierarchical indexing is not supported for simplicity.

We recommend deploying with 16x H20 using a combination of Tensor Parallelism and Expert Parallelism.

Compile and update sgl-kernel first.

```
cd sgl-kernel
python3 -m uv build --wheel --color=always --no-build-isolation \
    -Ccmake.define.SGL_KERNEL_ENABLE_SM90A=1 \
    -Ccmake.define.CMAKE_POLICY_VERSION_MINIMUM=3.5 \
    -Cbuild-dir=build .
pip3 install dist/sgl_kernel-0.3.21-cp310-abi3-linux_x86_64.whl --force-reinstall
```

Then launch the server.

```
python -m sglang.launch_server \
    --model meituan-longcat/LongCat-2.0-FP8 \
    --trust-remote-code \
    --host 0.0.0.0 \
    --port 13423 \
    --tp 16 \
    --ep 16 \
    --max-running-requests 64 \
    --mem-fraction-static 0.92 \
    --chunked-prefill-size 2048 \
    --nsa-prefill-backend fa3 \
    --kv-cache-dtype bfloat16 \
    --nnodes 2 \
    --node-rank 0 \
    --dist-init-addr 33.32.48.42:20000 \
    2>&1 | tee sgl.log
```

NPU

For NPU deployment, please refer to SGLang-FluentLLM.

Chat Template

We provide a chat template for LongCat-2.0 in the `tokenizer_config.json` file, which can be used to encode a list of messages into a single string for model input.

Here is a brief example of how to use the template:

```
from transformers import AutoTokenizer

tokenizer = AutoTokenizer.from_pretrained("meituan-longcat/LongCat-2.0",
trust_remote_code=True)

tools = [
    {
        "type": "function",
        "function": {
            "name": "func_add",
            "description": "Calculate the sum of two numbers",
            "parameters": {
                "type": "object",
                "properties": {
                    "x1": {"type": "number", "description": "The first number to add"},
                    "x2": {"type": "number", "description": "The second number to add"},
                },
                "required": ["x1", "x2"],
            },
        },
    },
    {
        "type": "function",
        "function": {
            "name": "func_multiply",
            "description": "Calculate the product of two numbers",
            "parameters": {
                "type": "object",
                "properties": {
                    "x1": {"type": "number", "description": "The first number to multiply"},
                    "x2": {"type": "number", "description": "The second number to
multiply"},
                },
                "required": ["x1", "x2"],
            },
        },
    },
]

messages = [
    {"role": "system", "content": "You are a helpful assistant."},
    {"role": "user", "content": "Calculate 1+1"},
    {
        "role": "assistant",
        "reasoning_content": "Calling func_add to calculate 1+1",
        # Note: unlike the standard OpenAI format, we expect `arguments` to be a dict rather
```

than a string.

```
    "tool_calls": [
        {"type": "function", "function": {"name": "func_add", "arguments": {"x1": 1,
"x2": 1}}},
    ],
},
{"role": "tool", "name": "func_add", "content": '{"ans": 2}'},
{"role": "assistant", "reasoning_content": "The result is 2", "content": "2"},
{"role": "user", "content": "Check your answer, is it correct?"},
]
```

thinking mode on

```
prompt_think = tokenizer.apply_chat_template(
    messages,
    tools=tools,
    tokenize=False,
    enable_thinking=True,
    add_generation_prompt=True
)
```

thinking mode on, keeping all reasoning content for better performance

```
prompt_full = tokenizer.apply_chat_template(
    messages,
    tools=tools,
    tokenize=False,
    enable_thinking=True,
    add_generation_prompt=True,
    save_reasoning_content=True
)
```

thinking mode off, for better token efficiency

```
prompt_no_think = tokenizer.apply_chat_template(
    messages,
    tools=tools,
    tokenize=False,
    enable_thinking=False,
    add_generation_prompt=True
)
```

License Agreement

The model weights are released under the MIT License.

Any contributions to this repository are licensed under the MIT License, unless otherwise stated. This license does not grant any rights to use Meituan trademarks or patents.

See the LICENSE file for the full license text.

Usage Considerations

This model has not been specifically designed or comprehensively evaluated for every possible downstream application.

Developers should take into account the known limitations of large language models, including performance variations across different languages, and carefully assess accuracy,

safety, and fairness before deploying the model in sensitive or high-risk scenarios. It is the responsibility of developers and downstream users to understand and comply with all applicable laws and regulations relevant to their use case, including but not limited to data protection, privacy, and content safety requirements.

Nothing in this Model Card should be interpreted as altering or restricting the terms of the MIT License under which the model is released.

Contact

Please contact us at longcat-team@meituan.com or open an issue if you have any questions.

Downloads last month

385

Safetensors

Model size

1.8T params

Tensor type

BF16

.

F32

Chat template

Files info

Inference Providers

NEW

Text Generation

This model isn't deployed by any Inference Provider.



13

Ask for provider support

Model tree for

meituan-longcat/LongCat-2.0

Quantizations

2 models

System theme

TOS

Privacy

About

Careers

Models

Datasets

Spaces

Pricing

Docs